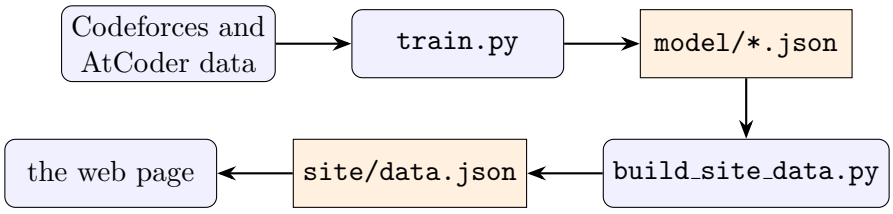
How the rating guesser works

Codeforces gives most problems a difficulty rating, but not all of them. New problems usually have none for a while, and AtCoder and CSES use different scales altogether. This tool tries to fill that gap. If a rating exists it just shows it. If not, it guesses one from things we can measure, like how many people solved the problem, and tells you how unsure it is.

This note explains what goes in, what the models do, and where they fall short.

1 The big picture

Training happens once on my computer (or in GitHub Actions) and needs `numpy` and `scikit-learn`. The website itself needs nothing: the trained models are saved as plain JSON, and every guess is worked out ahead of time and stored in one file that the page just looks things up in.



When you paste a problem, the page decides what to show:

1. Codeforces problem with a rating: show the rating.
2. Codeforces problem without one: show the model's guess and a range.
3. AtCoder task: convert AtCoder's difficulty to the Codeforces scale.
4. CSES: say there is nothing to go on.

2 The data

Everything comes from public sources. From Codeforces I take every problem with its tags, its rating (if it has one) and the number of people who solved it, plus the list of contests with their names and dates. From AtCoder Problems I take the difficulty estimate for each task.

The problems that already have a Codeforces rating are my training set. The model learns what rated problems look like, then applies that to the unrated ones.

3 What the model looks at

Each problem becomes a list of 53 numbers. The most useful one is the number of solvers s, on a log scale, because hard problems get solved by fewer people. But a raw count is misleading. A Div. 3 round has far more participants than a Div. 1 round, and a problem from five years ago has had more time to collect solves. So I also give the model:

- $\log \frac{s+1}{s_{\max}+1}$, the solves compared with the easiest problem in the same contest, and the problem's rank by solves;
- the position in the contest (A, B, C, ...) and how many problems the contest had;

- the contest type: Div. 1, 2, 3, 4, Educational, Global or other, worked out from the contest name;
- how old the contest is;
- the tags, one yes/no flag per tag.

One function builds these numbers for both training and the website, so the two can't drift apart.

4 The two models

Let y be the real rating and $\mathbf{z}$ the features after centering and scaling.

Ridge regression is a straight-line fit with a penalty that keeps the weights small:

$$\min_{\mathbf{w},b} \sum_i \big(y_i - \mathbf{w}^\top \mathbf{z}_i - b\big)^2 + \alpha \|\mathbf{w}\|^2. \tag{1}$$

The penalty matters because many features overlap (tags, contest types). I try a handful of values for α and keep the best.

Gradient boosting builds 300 small decision trees one after another, each one fixing the mistakes of those before it:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x}), \qquad \eta = 0.05. \tag{2}$$

Trees can pick up things a straight line can't, like solve counts meaning different things in different contest types.

Blending. The final guess mixes the two,

$$\hat{y} = w\, \hat{y}_{\text{boost}} + (1 - w)\, \hat{y}_{\text{ridge}}, \tag{3}$$

with w chosen from $0, 0.1, \ldots, 1$ to give the smallest average error on held-out data. The result is rounded to the nearest 100, like real ratings.

5 How I check it

Problems from the same contest are similar, so if some were in the training set and others in the test set, the score would look better than it deserves. I split by contest instead (5 folds), so the model is always tested on contests it has never seen. I report the average error, the share of guesses within 100 and 200 points, and compare against the dumb baseline of always guessing the median rating. A model that can't beat that isn't worth much.

The range shown with each guess is the 10th to 90th percentile of the model's past errors, which should cover the truth about 80% of the time. To make sure that claim holds, the training script computes the range on half the contests and counts how often it covers the other half.

6 AtCoder

No problem is rated on both sites, so there are no pairs to learn from. What I do instead is match percentiles. If an AtCoder problem is harder than 70% of regular AtCoder problems, I give it the Codeforces rating that is harder than 70% of regular Codeforces problems:

$$\hat{r}(d) = F_{\text{CF}}^{-1}\big(F_{\text{AT}}(d)\big). \tag{4}$$

I do this against two sets of Codeforces problems (all regular rounds, and Div. 2/3/4 plus Educational), take the average as the guess, and use the gap between them, plus 100, as the range.

This is only a rough guide. It assumes the two communities are similarly skilled, which isn't true, and I have no way to measure how wrong it is for a single problem.

7 Running it without scikit-learn

Training saves the scaling numbers, the ridge weights, and every tree as lists (left child, right child, feature, threshold, value). Predicting is then plain Python:

```
p=w*boost(x)+(1-w)*ridge(x)
return p,p+lo,p+hi
```

To use a tree, start at the top and go left if the feature is at most the threshold, right otherwise, until you hit a leaf.

8 Where it falls short

- A rating reflects how real contestants performed, and solve counts only explain part of that. Expect misses of a couple of hundred points on individual problems.
- Very new problems have few solves, so their guesses are rougher. The site says so.
- Contests and communities change over time, so the model should be retrained now and then. GitHub Actions does that weekly.
- The AtCoder mapping can't be validated, as explained above.
- CSES has no data at all, so no guess.

9 The files

`train.py`	downloads data, tests the models, saves them
`features.py`	turns a problem into numbers
`model_runtime.py`	predicts without scikit-learn
`build_site_data.py`	works out every guess ahead of time
`site/`	the web page
`estimator.py`	live version, used by the tests
`app.py`	local preview server